

System Architecture

Technical structure, data model, and integrations

Project	Course Enrollment Platform (courses: Digital Marketing, SEO, Content Writing, Web Development)
Version	1.0 (Draft for developer review)
Audience	Development team, project owner, and QA

1. Architecture Overview

The platform follows a standard three-tier web architecture: a responsive frontend, a secure API layer that holds all business logic, and a relational database, with a small set of third-party services for email, payments, and file storage.

Clients	Student browser (mobile / desktop) Admin browser Search engine crawlers
Frontend	Next.js (React) - server-rendered public pages for SEO, client-side dashboards for students and admin
API layer	Node.js REST API (NestJS or Express) - auth, courses, batches, enrollments, payments, admin, notifications
Data layer	PostgreSQL (primary data) Object storage (receipts, images, certificates) Redis (cache and rate limits, optional)
Services	Email (SMTP / Brevo) Payment gateway Analytics (GA4) WhatsApp click-to-chat Google Maps

Note on stack choice: This document recommends a custom Next.js + Node.js + PostgreSQL stack. If the developer proposes WordPress with an LMS plugin (for example Tutor LMS or LearnDash) plus WooCommerce, the modules, data model, and rules below still apply; only the implementation layer changes. The developer should state the chosen stack and the reason in writing before Phase 2.

2. Technology Stack

Layer	Recommended	Why
Frontend	Next.js, React, TypeScript, Tailwind CSS	Server rendering for SEO, fast pages, consistent styling
Backend	Node.js with NestJS (or Express), TypeScript	Structured modules, easy validation, one language across the stack
Database	PostgreSQL with Prisma ORM	Relational data (students, batches, payments), migrations, safe queries

Layer	Recommended	Why
Auth	JWT access + refresh tokens (httpOnly cookies), bcrypt hashing	Secure sessions and role checks
File storage	S3-compatible bucket (or server disk for V1)	Receipts, images, certificates
Email	SMTP or Brevo	Verification, receipts, reminders
PDF	Server-side PDF library (e.g. PDFKit)	Receipts and certificates
Hosting	VPS (Nginx + PM2 or Docker) or Vercel + managed database	Cost-effective, easy to scale
CI / Repo	GitHub with GitHub Actions	Automated lint, test, deploy

3. Modules

Module	Responsibility
Auth	Registration, login, email verification, password reset, role checks
Courses	Course catalog, curriculum, categories, instructors, FAQs
Batches	Start dates, timings, capacity, mode, assigned instructor
Enrollments	Create, confirm, cancel, waiting list, status history
Payments	Manual (receipt upload) and gateway payments, approval, invoices, refunds
Students	Profiles, dashboard data, certificates
Reviews	Submission by enrolled students, admin moderation
CMS	Testimonials, blog posts, banners, page meta tags
Notifications	Email templates and triggers
Admin	Dashboard stats, exports, user and role management, audit log

4. Data Model (Core Entities)

Entity	Key fields	Relationships
User	id, name, email, phone, password_hash, role, email_verified, status, created_at	Has many Enrollments
Course	id, title, slug, tagline, description, outcomes, level, duration, fee, image, status, seo_title, seo_description	Has many Modules, Batches, Reviews, FAQs
CourseModule	id, course_id, title, description, position	Belongs to Course
Instructor	id, name, bio, photo, social_links	Teaches many Batches
Batch	id, course_id, instructor_id, start_date, end_date, timing, mode, capacity, seats_taken, status	Belongs to Course; has many Enrollments
Enrollment	id, user_id, batch_id, status, total_fee, discount, created_at	Has many Payments
Payment	id, enrollment_id, amount, method, reference, receipt_url, status, approved_by, paid_at	Belongs to Enrollment
Review	id, user_id, course_id, rating, comment, status	Belongs to User and Course
Certificate	id, enrollment_id, code, issued_at, file_url	Belongs to Enrollment
BlogPost	id, title, slug, content, cover, seo fields, status, published_at	-
ContactMessage	id, name, email, phone, message, status, created_at	-
AuditLog	id, actor_id, action, entity, entity_id, created_at	Tracks admin actions

Status values

- **Enrollment:** pending, confirmed, waitlisted, cancelled, completed
- **Payment:** pending, approved, rejected, refunded
- **Batch:** upcoming, open, full, running, finished

5. API Design (REST, JSON)

All routes are prefixed with **/api/v1**. Responses use a consistent envelope: **{ success, data, error }**. Inputs are validated on the server for every route.

Area	Endpoints	Access
Auth	POST /auth/register, /auth/login, /auth/logout, /auth/verify-email, /auth/forgot-password, /auth/reset-password	Public
Courses	GET /courses, GET /courses/:slug	Public
Batches	GET /courses/:slug/batches	Public
Enrollments	POST /enrollments, GET /me/enrollments, PATCH /enrollments/:id/cancel	Student
Payments	POST /payments (upload receipt), GET /me/payments, GET /payments/:id/receipt	Student
Profile	GET/PATCH /me, GET /me/certificates	Student
Reviews	POST /courses/:id/reviews	Enrolled student
Contact	POST /contact	Public (rate limited)
Admin CRUD	/admin/courses, /admin/batches, /admin/instructors, /admin/faqs, /admin/testimonials, /admin/posts	Admin, Staff
Admin ops	PATCH /admin/enrollments/:id, PATCH /admin/payments/:id/approve reject, GET /admin/students/export, GET /admin/stats	Admin, Staff
Instructor	GET /instructor/batches, GET /instructor/batches/:id/students	Instructor

6. Key Flows

6.1 Enrollment and manual payment

- Student selects a course and an open batch and clicks Enroll.
- System checks: logged in, email verified, seats available, not already enrolled.
- Enrollment is created as **pending**; a seat is temporarily held for 48 hours.
- Student uploads a bank-transfer receipt (or pays through the gateway).
- Admin reviews the receipt and approves. Enrollment becomes **confirmed**, seat count updates, and receipt plus confirmation email are sent.
- If rejected or unpaid after the hold period, the seat is released and the student is notified.

6.2 Gateway payment

- Server creates a payment record and redirects to the gateway with a signed request.
- Gateway calls a webhook; the server verifies the signature and amount before marking the payment approved.
- The webhook is idempotent, so repeated calls never double-confirm an enrollment.

6.3 Certificate issue

- Admin marks an enrollment completed. The system generates a PDF with a unique verification code and emails it.
- A public page allows anyone to verify a certificate by its code.

7. Security Architecture

- HTTPS enforced with HSTS; secure, httpOnly, SameSite cookies.
- Passwords hashed with bcrypt (cost 12 or higher); never logged.
- Role-based access control checked on the server for every protected route.
- Input validation and output encoding on all forms; parameterized queries only.

- CSRF protection, strict CORS, security headers (Helmet), and rate limiting on login, register, and contact endpoints.
- CAPTCHA on registration and contact forms.
- File uploads limited by type (JPG, PNG, PDF) and size (5 MB), scanned by name and MIME, stored outside the web root.
- Secrets in environment variables only; no secrets in the repository.
- Audit log for admin actions on payments, enrollments, and user roles.

8. Hosting, Environments, and Operations

Item	Approach
Environments	Local, Staging (for owner review), Production
Deployment	Git push to main triggers CI (lint, tests, build) and deploys to staging; production deploy after approval
Backups	Daily automated database backup, 30-day retention, monthly restore test; uploads backed up weekly
Monitoring	Uptime monitor, error tracking (e.g. Sentry), server log rotation
Caching	CDN for static assets; cache course list and course pages; revalidate on admin update
Scaling path	Vertical scaling first; add Redis and a read replica, or move to containers, if traffic grows

9. Recommended Folder Structure

```

/apps/web      - Next.js frontend (pages, components, styles)
/apps/api      - Backend (modules: auth, courses, batches, enrollments, payments, admin)
/packages/ui    - Shared design components and tokens
/prisma        - Schema and migrations
/docs          - These five documents, API docs, admin guide
/.github       - CI workflows

```